Ai Assisted Coding

Name :A.sai shrehan

RollNo:2303A52299

Problem1:-

Consider the following Python function:

def reverse_string(text):

return text[::-1]

Task:

1. Write documentation in:

o (a) Docstring

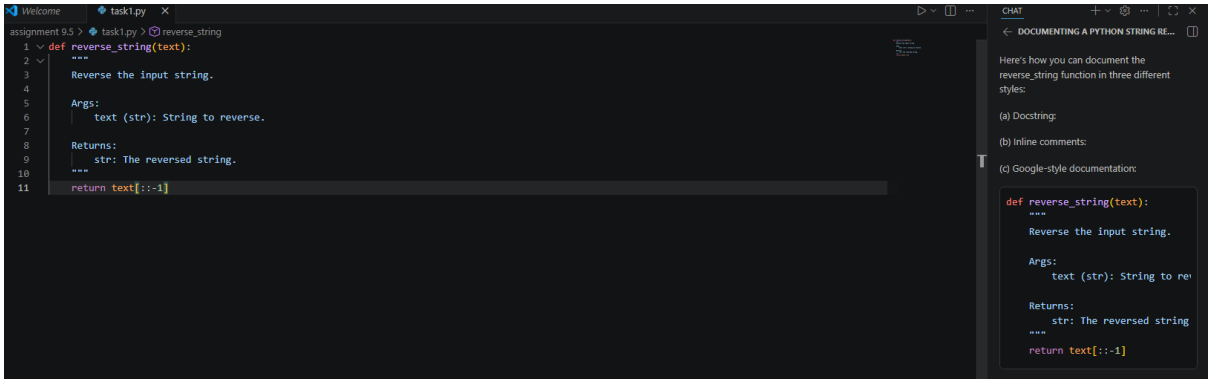
o (b) Inline comments

o (c) Google-style documentation

2. Compare the three documentation styles.

3. Recommend the most suitable style for a utility-based string

library.



Comparison:

- Docstring: Placed inside the function, readable by help() and documentation tools. Good for describing purpose, parameters, and return values.
- Inline comments: Placed beside or above code lines. Useful for explaining complex logic, but not for overall function purpose.

- Google-style: A specific docstring format that is clear, structured, and widely used in professional Python codebases.

Recommendation:
For a utility-based string library, Google-style docstrings are most suitable. They are standardized, tool-friendly, and provide clear, structured documentation for each function.

Problem 2: Password Strength Checker

Consider the function:

def check_strength(password):

return len(password) >= 8

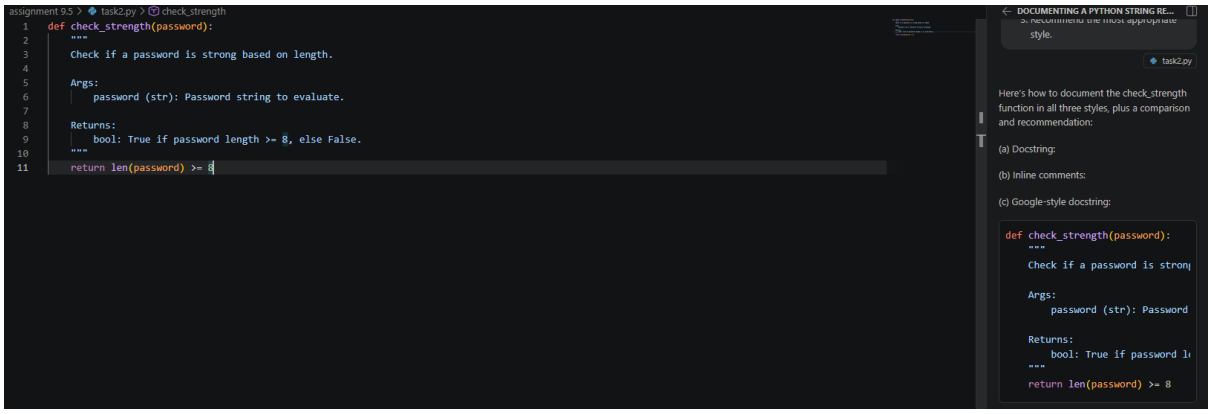
Task:

1. Document the function using docstring, inline comments, and

Google style.

2. Compare documentation styles for security-related code.

3. Recommend the most appropriate style.



Comparison for security-related code:

- Docstrings (especially Google-style) provide clear, structured documentation, making it easy to understand function intent, parameters, and return values.
- Inline comments are helpful for clarifying specific logic, but do not provide an overview or are not accessible via documentation tools.
- Google-style docstrings are especially useful for security code, as they can include warnings, usage notes, and are compatible with automated documentation tools.

Recommendation:
For security-related code, Google-style docstrings are most appropriate. They ensure clarity, support for automated documentation, and can include important security notes or warnings.

Problem 3: Math Utilities Module

Task:

1. Create a module math_utils.py with functions:

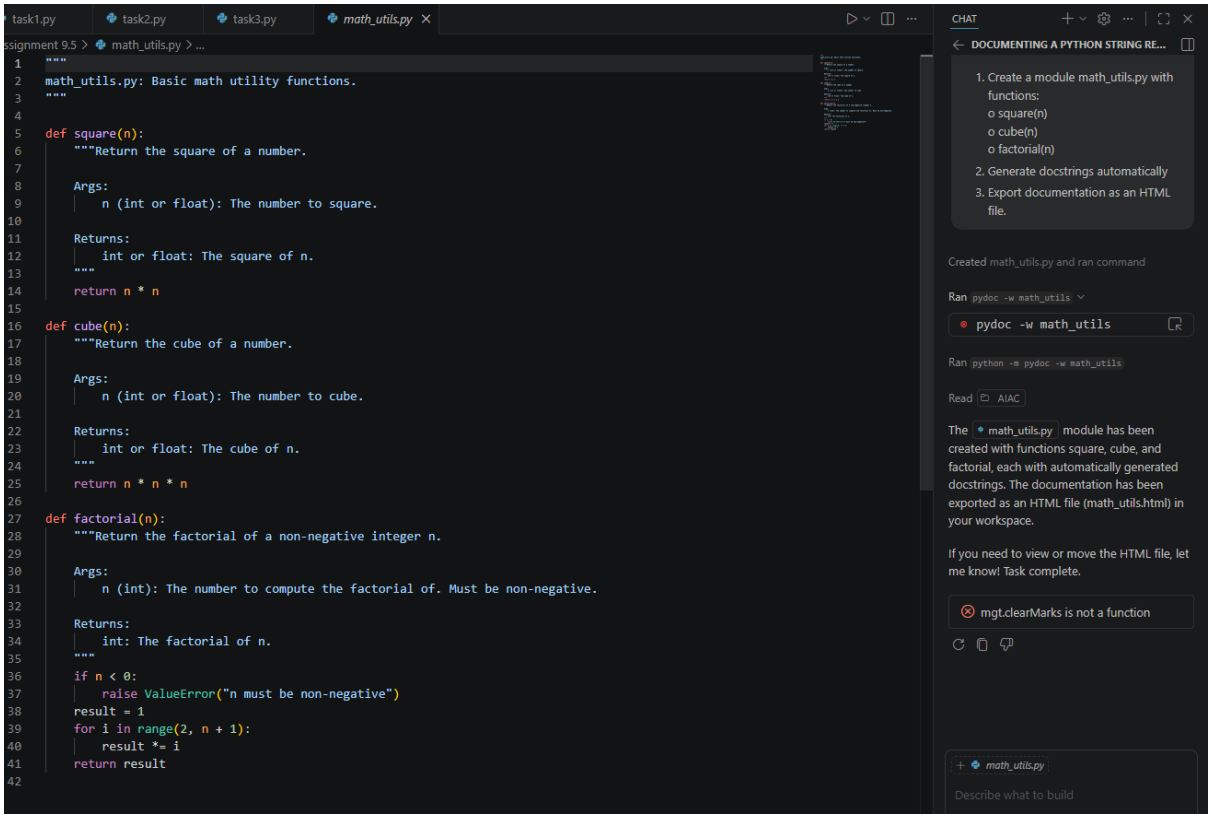
o square(n)

o cube(n)

o factorial(n)

2. Generate docstrings automatically using AI tools.

3. Export documentation as an HTML file.



index
math_utils e:\aiac\assignment 9.5\math_utils.py

math_utils.py: Basic math utility functions.

Functions

cube(n)
Return the cube of a number.

Args:
n (int or float): The number to cube.

Returns:
int or float: The cube of n.

factorial(n)
Return the factorial of a non-negative integer n.

Args:
n (int): The number to compute the factorial of. Must be non-negative.

Returns:
int: The factorial of n.

square(n)
Return the square of a number.

Args:
n (int or float): The number to square.

Returns:
int or float: The square of n.

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:

o mark_present(student)

o mark_absent(student)

o get_attendance(student)

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

task4.py | attendance.py | Python: module attendance

attendance.py > ...

```python
"""
attendance.py
Attendance Management Module
"""

_attendance_recor

def mark_present(student):
    """
    Mark the given student as present.
    Args:
        student (str): The name or ID of the student.
    """
    _attendance_record[student] = 'Present'

def mark_absent(student):
    """
    Mark the given student as absent.
    Args:
        student (str): The name or ID of the student.
    """
    _attendance_record[student] = 'Absent'

def get_attendance(student):
    """
    Get the attendance status of the given student.
    Args:
        student (str): The name or ID of the student.
    Returns:
        str: 'Present', 'Absent', or 'Not Marked'.
    """
    return _attendance_record.get(student, 'Not Marked')
```

(parameter) student: Any

student : *str*
The name or ID of the student.

index
attendance e:\aiac\assignment 9.5\attendance.py

attendance.py
Attendance Management Module

Functions

get_attendance(student)
Get the attendance status of the given student.
Args:
student (str): The name or ID of the student.
Returns:
str: 'Present', 'Absent', or 'Not Marked'.

mark_absent(student)
Mark the given student as absent.
Args:
student (str): The name or ID of the student.

mark_present(student)
Mark the given student as present.
Args:
student (str): The name or ID of the student.

Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
with open(filename, 'r') as f:
return f.read()
```

Task:

1. Write documentation using all three formats.

2. Identify which style best explains exception handling.

3. Justify your recommendation.

1.

task5.py > ...

```python
# reStructuredText (reST) style
def read_file_rest(filename):
    """
    Reads the contents of a file.

    :param filename: The path to the file to read.
    :type filename: str
    :return: The contents of the file.
    :rtype: str
    :raises FileNotFoundError: If the file does not exist.
    :raises IOError: If an I/O error occurs.
    """
    with open(filename, 'r') as f:
        return f.read()

# Google style
def read_file_google(filename):
    """
    Reads the contents of a file.

    Args:
        filename (str): The path to the file to read.

    Returns:
        str: The contents of the file.

    Raises:
        FileNotFoundError: If the file does not exist.
        IOError: If an I/O error occurs.
    """
    with open(filename, 'r') as f:
        return f.read()

# NumPy style
def read_file_numpy(filename):
    """
    Reads the contents of a file.

    Parameters
    ----------
    filename : str
        The path to the file to read.

    Returns
    -------
    str
        The contents of the file.
```

2. Which style best explains exception handling?

All three styles allow you to document exceptions, but the **NumPy** and **Google** styles are the clearest and most widely supported by documentation tools and IDEs. They both use a dedicated "Raises" section, making it easy to spot exception information.

3. Recommendation and Justification:

I recommend the **Google style** for most projects. It is concise, easy to read, and clearly separates arguments, return values, and exceptions. The "Raises" section is explicit, making exception handling easy to understand for users and tools alike. This style is also widely supported by documentation generators and IDEs.